

Introduction to Processor Architecture (IPA)

Spring 2026 — Mid-Semester Examination

ANSWER KEY

Total Marks: 60 | Duration: 90 Minutes | Course: CS 2.203

Question 1: Pipelined RISC-V Datapath & Hazard Handling [25 Marks]

(a)(i) RAW Data Dependencies [5 Marks]

The following table lists all Read-After-Write (RAW) data dependencies across instructions I1 through I11. Since the branch at I4 is taken, the actual execution path is I1 → I2 → I3 → I4 → I9 → I10 → I11 (with I5 and I6 fetched speculatively but flushed). Dependencies involving I5 and I6 are included since they appear in the listed instruction sequence.

#	Producer	Consumer	Register	Notes
1	I1: lw x1, 0(x10)	I2: add x3, x1, x2	x1	Distance 1 (load)
2	I2: add x3, x1, x2	I3: sw x3, 4(x10)	x3	Distance 1
3	I2: add x3, x1, x2	I4: bne x3, x0, TARGET	x3	Distance 2
	I1: lw x1, 0(x10)	I9: sub x8, x1, x3	x1	Far (across branch)
	I2: add x3, x1, x2	I9: sub x8, x1, x3	x3	Far (across branch)
6	I9: sub x8, x1, x3	I10: and x9, x8, x2	x8	Distance 1
7	I10: and x9, x8, x2	I11: sw x9, 12(x10)	x9	Distance 1
	I5: addi x12, x10, 8	I6: lw x13, 0(x12)	x12	Both flushed

Note: x0 in I4 is hardwired to zero and is never written, so bne x3, x0 does not create a RAW dependency on x0. Similarly, x10 and x2 are only read (never written) in this sequence, so they produce no RAW dependencies.

(a)(ii) Hazard Classification [5 Marks]

#	Dependency	Type	Category	Explanation
1	I1 → I2 (x1)	Load-use hazard	(C) Stall	I1 (lw) data ready end of MEM; I2 needs it start of EX. 1 stall required.
2	I2 → I3 (x3)	EX/MEM fwd	(A)	I2 result available at end of EX, forwarded to I3's EX (next cycle).
3	I2 → I4 (x3)	MEM/WB fwd	(B)	I2 is 2 stages ahead of I4 in the actual pipeline; data forwarded from MEM/WB.

	I1 → I9 (x1)	No hazard	—	I1 completes WB long before I9 reaches ID. Register file read suffices.
	I2 → I9 (x3)	No hazard	—	I2 completes WB before I9 reaches ID. No forwarding needed.
6	I9 → I10 (x8)	EX/MEM fwd	(A)	I9 result at end of EX, forwarded to I10's EX (next cycle).
7	I10 → I11 (x9)	EX/MEM fwd	(A)	I10 result at end of EX, forwarded to I11's EX (next cycle).
	I5 → I6 (x12)	EX/MEM fwd	(A)	Would use EX/MEM fwd, but both instructions are flushed.

Why load-use hazards cannot be resolved by forwarding alone: A load instruction (lw) reads data from memory at the END of its MEM stage. However, the dependent instruction needs this value at the BEGINNING of its EX stage. In a standard 5-stage pipeline, MEM occurs after EX in the same clock cycle, so the data is simply not available in time. Even with a MEM-to-EX forwarding path, the consumer instruction in EX would need data that is only produced at the end of that same cycle's MEM stage — a timing impossibility. Therefore, one stall cycle must be inserted so the load's MEM output (now in the MEM/WB pipeline register) can be forwarded to the consumer's EX stage in the following cycle.

(b) Pipeline Execution Diagram [15 Marks]

The complete cycle-by-cycle execution diagram is shown below. EX* denotes that forwarding is used in that stage. S = stall bubble, [F] = flushed (squashed due to branch misprediction).

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1: lw	IF	ID	EX	MEM	WB									
I2: add		IF	ID	S	EX*	MEM	WB							
I3: sw			IF	S	ID	EX*	MEM	WB						
I4: bne					IF	ID	EX*	MEM	WB					
I5: addi						IF	ID	[F]						
I6: lw							IF	[F]						
I9: sub							IF	ID	EX	MEM	WB			
I10: and								IF	ID	EX*	MEM	WB		
I11: sw									IF	ID	EX*	MEM	WB	

Key events in the pipeline:

Cycle 3–4 (Load-use stall): I1 (lw) is in EX at cycle 3 and the hazard unit detects that I2 reads x1. A stall bubble is inserted: I2 stays in ID, I3 stays in IF, and EX gets a NOP. In cycle 5, I2 enters EX and receives x1 from the MEM/WB pipeline register (forwarded from I1's MEM stage output at end of cycle 4).

Cycle 6 (I2 → I3 forwarding): I2 is in MEM, I3 is in EX. I3 reads x3 which was produced by I2. The value is forwarded from the EX/MEM pipeline register (I2's EX result from cycle 5).

Cycle 7 (I2 → I4 forwarding & branch resolution): I4 is in EX and reads x3 from I2. I2 is in WB (data in MEM/WB register from cycle 6), so MEM/WB forwarding is used. At the end of cycle 7, the branch is resolved as TAKEN. Since we predicted not-taken, I5 (in ID) and I6 (in IF) must be flushed.

Cycle 8 (Branch penalty): I5 and I6 are squashed (shown as [F] in cycles 7–8). The correct target I9 begins fetching in cycle 8. This constitutes a 2-cycle branch misprediction penalty.

Cycles 11–12 (I9 → I10 and I10 → I11 forwarding): I9's result (x8) is forwarded via EX/MEM to I10 in cycle 11. I10's result (x9) is forwarded via EX/MEM to I11 in cycle 12.

Total clock cycles from I1's IF to I11's WB = 14 cycles

Breakdown: 11 base cycles (for 7 completing instructions in 5-stage pipeline: $5 + 6 = 11$) + 1 load-use stall + 2 branch misprediction penalty = 14 cycles.

Question 2: Cache Memory Design & Performance Analysis [25 Marks]

(a)(i) Cache Parameters [5 Marks]

Given: Cache Size = 16 KB, Block Size = 64 bytes, Associativity = 4-way, Address Space = 32 bits

Total number of cache blocks:

$$\text{Total Blocks} = \text{Cache Size} / \text{Block Size} = 16,384 / 64 = 256 \text{ blocks}$$

Number of sets:

$$\text{Number of Sets} = \text{Total Blocks} / \text{Associativity} = 256 / 4 = 64 \text{ sets}$$

Address field breakdown (32-bit address):

$$\text{Offset bits} = \log_2 (\text{Block Size}) = \log_2 (64) = 6 \text{ bits}$$

$$\text{Index bits} = \log_2 (\text{Number of Sets}) = \log_2 (64) = 6 \text{ bits}$$

$$\text{Tag bits} = 32 - 6 - 6 = 20 \text{ bits}$$

Tag	Index	Offset
Bits [31:12] — 20 bits	Bits [11:6] — 6 bits	Bits [5:0] — 6 bits

(a)(ii) Cache Organization Diagram [10 Marks]

The diagram should illustrate one complete set of the 4-way set-associative cache with the following components:

Structure per set (one of 64 sets):

Each set contains 4 ways. Each way stores: 1 Valid bit (V), 1 Dirty bit (D), a 20-bit Tag field, and a 64-byte (512-bit) Data block. Overhead per set is $4 * (1 + 1 + 20) = 88 \text{ bits} + 5 \text{ bits for LRU FSM} = 93 \text{ bits}$, for 64 sets, the overhead is $64 \times 93 = 5952 \text{ bits}$

Hit/Miss detection logic:

The 6-bit index field selects one set. For each of the 4 ways, the stored tag is compared with the incoming 20-bit tag using a comparator ($=$). Each comparator output is ANDed with the corresponding valid bit (V). The four AND gate outputs feed into a single OR gate: if any output is 1, it is a cache HIT; otherwise it is a MISS.

Data output multiplexer:

A 4-to-1 MUX selects the data from the hitting way. The 6-bit block offset selects the specific byte/word from the 64-byte block.

LRU tracking mechanism:

Each set maintains LRU state bits to track the access ordering of the 4 ways. A common hardware implementation uses a 4×4 upper-triangular bit matrix (6 bits per set). On each access, the row bits for the accessed way are set to 1 and the column bits are cleared to 0. The LRU victim is the way whose row is all 0s and column is all 1s.

(a)(iii) Storage Overhead Calculation [5 Marks]**Per-block overhead:**

Tag: 20 bits, Valid: 1 bit, Dirty: 1 bit → 22 bits per block

Total for all blocks: $256 \times 22 = 5,632$ bits

LRU overhead per set:

Using the bit-matrix approach for 4-way LRU: $n(n-1)/2 = 4 \times 3/2 = 6$ bits per set

Total for all sets: $64 \times 6 = 384$ bits

Total overhead:

Total overhead = $5,632 + 384 = 6,016$ bits

Data storage:

Data = 16 KB = $16,384 \times 8 = 131,072$ bits

Total cache storage (data + overhead):

Total = $131,072 + 6,016 = 137,088$ bits

Overhead percentage = $(6,016 / 137,088) \times 100 \approx 4.39\%$

Note: If using $\lceil \log_2 (4!) \rceil = \lceil \log_2 (24) \rceil = 5$ bits per set for LRU instead of the bit-matrix approach, total LRU overhead = $64 \times 5 = 320$ bits, total overhead = **5,952 bits**, and overhead percentage $\approx 4.34\%$. Both approaches are acceptable.

(b) Multi-Level Cache Performance (AMAT) [5 Marks]**Given:**

L1 D-cache: Hit Time = 1 cycle, Local Miss Rate = 6.25%

L2 (unified): Hit Time = 10 cycles, Local Miss Rate = 15%

Main Memory: Access Time = 100 cycles

Recursive AMAT formula:

$$\text{AMAT} = \text{Hit_L1} + \text{MissRate_L1} \times (\text{Hit_L2} + \text{MissRate_L2} \times \text{Penalty_Mem})$$

Substituting values:

$$\text{AMAT} = 1 + 0.0625 \times (10 + 0.15 \times 100)$$

$$\text{AMAT} = 1 + 0.0625 \times (10 + 15)$$

$$\text{AMAT} = 1 + 0.0625 \times 25$$

$$\text{AMAT} = 1 + 1.5625$$

AMAT = 2.5625 cycles

Question 3: Multiple Choice Questions on Cache Memory [10 Marks]

Q	Answer	Explanation
(a)	B	Spatial locality. The cache fetches a 64-byte block on the first access, and subsequent accesses to nearby array elements hit within the same block. This exploits the principle that data near a recently accessed address is likely to be accessed soon — the definition of spatial locality. (Temporal locality would apply if the same address were re-accessed, which is not the scenario described.)
(b)	C	Conflict misses. In a direct-mapped cache with 4 lines, addresses 0 and 8 both map to line 0 (since $0 \bmod 4 = 0$ and $8 \bmod 4 = 0$). They continuously evict each other, causing every access to miss after the first. Although compulsory misses account for the very first access to each block, the repeated misses on 0 and 8 are due to conflicts — they map to the same cache line despite sufficient total cache capacity.
(c)	C	Increase associativity. Thrashing in a direct-mapped cache occurs because two blocks map to the same line and continuously evict each other. A 2-way (or higher) set-associative cache allows multiple blocks to coexist in the same set, directly eliminating this conflict. Increasing block size (A) does not help since the blocks map to different addresses. Write-back policy (B) only affects write traffic, not hit rate. Random replacement (D) does not help in a direct-mapped cache (only 1 way).
(d)	B	Write-back reduces memory bus traffic because dirty blocks are only written to memory upon eviction, not on every write. Write-allocate complements this by loading the block into the cache on a write miss, allowing subsequent writes (and reads) to that block to hit in the cache — exploiting temporal locality. Option A is incorrect (write-back makes memory inconsistent until eviction). Option C is incorrect (write-back requires a dirty bit). Option D is incorrect (write-through with no-write-allocate is compatible with FENCE).
(e)	C	20 bits. With a 32-bit address, 128 entries (direct-mapped $\rightarrow$ 128 sets), and 32-byte blocks: Offset = $\log_2(32) = 5$ bits; Index = $\log_2(128) = 7$ bits; Tag = $32 - 5 - 7 = 20$ bits.

— End of Answer Key —